

TP Noté – Terraform sur Azure

Déploiement de 2 VMs avec Load Balancer

Nom : KHITARIDZE George

Date : 23/02/2026

GitHub : <https://github.com/GeorgeKhita/TP-Terraform>

Partie 1 – Structure du projet et provider

Le projet Terraform a été structuré selon les conventions vues en cours, avec les fichiers suivants dans le répertoire tp-terraform-azure :

- versions.tf - Contraintes de version Terraform ($\geq 1.5.0$) et provider azurearm (~ 4.0)
- provider.tf - Configuration du provider Azure avec authentification via az cli
- variables.tf - Déclaration des variables location, prefix et subscription_id
- main.tf - Déclaration de toutes les ressources Azure
- outputs.tf - Exposition des valeurs créées (IDs, IPs, noms)

terraform init

La commande terraform init a été exécutée pour initialiser le répertoire de travail et télécharger le provider azurearm v4.x.

```
C:\Projects\tp-terraform-azure>terraform init
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/azurerm versions matching "~> 4.0"...
- Installing hashicorp/azurerm v4.61.0...
- Installed hashicorp/azurerm v4.61.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
C:\Projects\tp-terraform-azure>
```

Partie 2 – Réseau : Resource Group, VNET et Subnet

Les ressources réseau de base ont été créées dans la région France Central (la région West Europe étant restreinte pour les comptes étudiants) :

- Resource Group : tp-azure-rg, avec les tags environment = "tp" et managed_by = "terraform"
- Virtual Network : tp-azure-vnet, espace d'adressage 10.0.0.0/16
- Subnet : tp-azure-subnet, plage d'adresses 10.0.1.0/24

Les outputs suivants ont été configurés dans outputs.tf :

- ID du Resource Group
- Nom du VNET

- ID du Subnet

terraform plan

La commande terraform plan a permis de vérifier la configuration avant tout déploiement réel sur Azure.

```
C:\Projects\tp-terraform-azure>terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# azurerm_lb.lb will be created
+ resource "azurerm_lb" "lb" {
  + id                        = (known after apply)
  + location                 = "francecentral"
  + name                     = "tp-azure-lb"
  + private_ip_address       = (known after apply)
  + private_ip_addresses     = (known after apply)
  + public_ip_address_id     = (known after apply)
  + resource_group_name      = "tp-azure-rg"
  + sku                      = "Standard"
  + sku_tier                 = "Regional"
  + subnet_id                = (known after apply)

  + frontend_ip_configuration {
    + gateway_load_balancer_frontend_ip_configuration_id = (known after apply)
    + id                                                  = (known after apply)
    + inbound_nat_rules                                = (known after apply)
    + load_balancer_rules                              = (known after apply)
    + name                                               = "frontend"
    + outbound_rules                                    = (known after apply)
    + private_ip_address                               = (known after apply)
    + private_ip_address_allocation                    = (known after apply)
    + private_ip_address_version                       = (known after apply)
    + public_ip_address_id                             = (known after apply)
    + public_ip_prefix_id                             = (known after apply)
  }
}
```

...

```
# azurerm_virtual_network.vnet will be created
+ resource "azurerm_virtual_network" "vnet" {
  + address_space = [
    + "10.0.0.0/16",
  ]
  + dns_servers    = (known after apply)
  + guid          = (known after apply)
  + id            = (known after apply)
  + location      = "francecentral"
  + name          = "tp-azure-vnet"
  + private_endpoint_vnet_policies = "Disabled"
  + resource_group_name = "tp-azure-rg"
  + subnet        = (known after apply)
}

Plan: 16 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ lb_public_ip      = (known after apply)
+ resource_group_id = (known after apply)
+ subnet_id         = (known after apply)
+ vnet_name         = "tp-azure-vnet"

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform
apply" now.

C:\Projects\tp-terraform-azure>
```

Partie 3 – Sécurité : Network Security Group

Un Network Security Group (tp-azure-nsg) a été créé avec 3 règles entrantes pour sécuriser les VMs tout en autorisant le trafic nécessaire :

- allow-ssh (priorité 100) : autorise le port 22 en TCP
- allow-http (priorité 200) : autorise le port 80 en TCP pour Nginx
- deny-all-inbound (priorité 4096) : bloque tout autre trafic entrant

Le NSG a été associé au subnet via une ressource `azurerm_subnet_network_security_group_association`.

Partie 4 – Machines virtuelles

Deux machines virtuelles Linux Ubuntu 22.04 LTS (taille Standard_B1s) ont été déployées avec les caractéristiques suivantes :

- Nom : tp-azure-vm-1 et tp-azure-vm-2
- Authentification par clé SSH (azureuser)
- Disque OS : Standard_LRS, 30 Go
- Image : Canonical, Ubuntu Server 22.04 LTS Gen2

Installation automatique de Nginx (custom_data)

L'attribut `custom_data` a été utilisé pour installer et configurer Nginx automatiquement au premier démarrage de chaque VM. Le script bash encodé en base64 réalise les opérations suivantes :

1. Mise à jour des paquets (`apt-get update`)
2. Installation de Nginx (`apt-get install -y nginx`)
3. Création d'une page HTML personnalisée affichant le nom de la VM
4. Démarrage et activation du service Nginx

Partie 5 – Load Balancer

Un Load Balancer public Standard a été mis en place pour distribuer le trafic HTTP entre les 2 VMs. Les ressources créées sont :

- IP publique statique Standard : tp-azure-lb-pip
- Load Balancer : tp-azure-lb (SKU Standard)

- Backend Address Pool : tp-azure-backend-pool, avec les 2 NICs associées
- Health Probe HTTP : tp-azure-http-probe (port 80, path /)
- Load Balancing Rule : tp-azure-http-rule (port 80 → 80)

L'IP publique du Load Balancer a été exposée dans les outputs pour faciliter les tests.

Partie 6 - Validation et nettoyage

terraform apply

Le déploiement a été effectué avec terraform apply. Au total, 16 ressources ont été créées avec succès sur Azure en région France Central.

```
C:\Projects\tp-terraform-azure>terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# azure_rm_lb will be created
+ resource "azure_rm_lb" "lb" {
  + id                        = (known after apply)
  + location                 = "francecentral"
  + name                     = "tp-azure-lb"
  + private_ip_address      = (known after apply)
  + private_ip_addresses    = (known after apply)
  + public_ip_address_id    = (known after apply)
  + resource_group_name     = "tp-azure-rg"
  + sku                      = "Standard"
  + sku_tier                 = "Regional"
  + subnet_id               = (known after apply)

  + frontend_ip_configuration {
    + gateway_load_balancer_frontend_ip_configuration_id = (known after apply)
    + id                                                  = (known after apply)
    + inbound_nat_rules                                = (known after apply)
    + load_balancer_rules                              = (known after apply)
    + name                                               = "frontend"
    + outbound_rules                                   = (known after apply)
    + private_ip_address                               = (known after apply)
    + private_ip_address_allocation                    = (known after apply)
    + private_ip_address_version                       = (known after apply)
    + public_ip_address_id                             = (known after apply)
    + public_ip_prefix_id                             = (known after apply)
  }
}
```

...

```
azurerm_linux_virtual_machine.vm1: Still creating... [00m40s elapsed]
azurerm_linux_virtual_machine.vm1: Creation complete after 48s [id=/subscriptions/cb514673-1b19-46b0-b454-5b670918d9e4/resourceGroups/tp-azure-rg/providers/Microsoft.Compute/virtualMachines/tp-azure-vm-1]

Apply complete! Resources: 16 added, 0 changed, 0 destroyed.

Outputs:
lb_public_ip = "20.188.56.234"
resource_group_id = "/subscriptions/cb514673-1b19-46b0-b454-5b670918d9e4/resourceGroups/tp-azure-rg"
subnet_id = "/subscriptions/cb514673-1b19-46b0-b454-5b670918d9e4/resourceGroups/tp-azure-rg/providers/Microsoft.Network/virtualNetworks/tp-azure-vnet/subnets/tp-azure-subnet"
vnet_name = "tp-azure-vnet"

C:\Projects\tp-terraform-azure>
```

Test d'accès via le Load Balancer

L'accès au serveur web a été testé via l'IP publique du Load Balancer. La commande curl a été répétée plusieurs fois pour confirmer que les 2 VMs répondent alternativement.

```
C:\Projects\tp-terraform-azure>curl http://20.188.56.234
<h1>Hello from tp-azure-vm-1</h1>

C:\Projects\tp-terraform-azure>curl http://20.188.56.234
<h1>Hello from tp-azure-vm-2</h1>

C:\Projects\tp-terraform-azure>curl http://20.188.56.234
<h1>Hello from tp-azure-vm-1</h1>

C:\Projects\tp-terraform-azure>curl http://20.188.56.234
<h1>Hello from tp-azure-vm-2</h1>

C:\Projects\tp-terraform-azure>curl http://20.188.56.234
<h1>Hello from tp-azure-vm-1</h1>

C:\Projects\tp-terraform-azure>curl http://20.188.56.234
<h1>Hello from tp-azure-vm-2</h1>

C:\Projects\tp-terraform-azure>curl http://20.188.56.234
<h1>Hello from tp-azure-vm-1</h1>

C:\Projects\tp-terraform-azure>curl http://20.188.56.234
<h1>Hello from tp-azure-vm-2</h1>

C:\Projects\tp-terraform-azure>
```

terraform destroy

Une fois les tests effectués et les captures réalisées, toutes les ressources Azure ont été supprimées avec terraform destroy pour ne pas consommer le crédit Azure étudiant inutilement.

```
C:\Projects\tp-terraform-azure>terraform destroy
azurerm_resource_group.rg: Refreshing state... [id=/subscriptions/cb514673-1b19-46b0-b454-5b670918d9e4/resourceGroups/tp-azure-rg]
azurerm_public_ip.lb_pip: Refreshing state... [id=/subscriptions/cb514673-1b19-46b0-b454-5b670918d9e4/resourceGroups/tp-azure-rg/providers/Microsoft.Network/publicIPAddresses/tp-azure-lb-pip]
azurerm_public_ip.vm2_pip: Refreshing state... [id=/subscriptions/cb514673-1b19-46b0-b454-5b670918d9e4/resourceGroups/tp-azure-rg/providers/Microsoft.Network/publicIPAddresses/tp-azure-vm2-pip]
azurerm_public_ip.vm1_pip: Refreshing state... [id=/subscriptions/cb514673-1b19-46b0-b454-5b670918d9e4/resourceGroups/tp-azure-rg/providers/Microsoft.Network/publicIPAddresses/tp-azure-vm1-pip]
azurerm_network_security_group.nsg: Refreshing state... [id=/subscriptions/cb514673-1b19-46b0-b454-5b670918d9e4/resourceGroups/tp-azure-rg/providers/Microsoft.Network/networkSecurityGroups/tp-azure-nsg]
azurerm_virtual_network.vnet: Refreshing state... [id=/subscriptions/cb514673-1b19-46b0-b454-5b670918d9e4/resourceGroups/tp-azure-rg/providers/Microsoft.Network/virtualNetworks/tp-azure-vnet]
azurerm_lb.lb: Refreshing state... [id=/subscriptions/cb514673-1b19-46b0-b454-5b670918d9e4/resourceGroups/tp-azure-rg/providers/Microsoft.Network/loadBalancers/tp-azure-lb]
azurerm_subnet.subnet: Refreshing state... [id=/subscriptions/cb514673-1b19-46b0-b454-5b670918d9e4/resourceGroups/tp-azure-rg/providers/Microsoft.Network/virtualNetworks/tp-azure-vnet/subnets/tp-azure-subnet]
azurerm_lb_backend_address_pool.backend_pool: Refreshing state... [id=/subscriptions/cb514673-1b19-46b0-b454-5b670918d9e4/resourceGroups/tp-azure-rg/providers/Microsoft.Network/loadBalancers/tp-azure-lb/backendAddressPools/tp-azure-backend-pool]
azurerm_lb_probe.http_probe: Refreshing state... [id=/subscriptions/cb514673-1b19-46b0-b454-5b670918d9e4/resourceGroups/tp-azure-rg/providers/Microsoft.Network/loadBalancers/tp-azure-lb/probes/tp-azure-http-probe]
azurerm_subnet_network_security_group_association.nsg_assoc: Refreshing state... [id=/subscriptions/cb514673-1b19-46b0-b454-5b670918d9e4/resourceGroups/tp-azure-rg/providers/Microsoft.Network/virtualNetworks/tp-azure-vnet/subnets/tp-azure-subnet]
azurerm_network_interface.nic1: Refreshing state... [id=/subscriptions/cb514673-1b19-46b0-b454-5b670918d9e4/resourceGroups/tp-azure-rg/providers/Microsoft.Network/networkInterfaces/tp-azure-nic-1]
```

...

```
azurerm_public_ip.vm2_pip: Destroying... [id=/subscriptions/cb514673-1b19-46b0-b454-5b670918d9e4/resourceGroups/tp-azure-rg/providers/Microsoft.Network/publicIPAddresses/tp-azure-vm2-pip]
azurerm_public_ip.vm1_pip: Destruction complete after 12s
azurerm_public_ip.vm2_pip: Still destroying... [id=/subscriptions/cb514673-1b19-46b0-b454-5b670918d9e4/resourceGroups/tp-azure-rg/providers/Microsoft.Network/publicIPAddresses/tp-azure-vm2-pip, 00m10s elapsed]
azurerm_subnet.subnet: Still destroying... [id=/subscriptions/cb514673-1b19-46b0-b454-5b670918d9e4/resourceGroups/tp-azure-rg/providers/Microsoft.Network/subnets/tp-azure-subnet, 00m10s elapsed]
azurerm_public_ip.vm2_pip: Destruction complete after 12s
azurerm_subnet.subnet: Destruction complete after 12s
azurerm_virtual_network.vnet: Destroying... [id=/subscriptions/cb514673-1b19-46b0-b454-5b670918d9e4/resourceGroups/tp-azure-rg/providers/Microsoft.Network/virtualNetworks/tp-azure-vnet]
azurerm_virtual_network.vnet: Still destroying... [id=/subscriptions/cb514673-1b19-46b0-b454-5b670918d9e4/resourceGroups/tp-azure-rg/providers/Microsoft.Network/virtualNetworks/tp-azure-vnet, 00m10s elapsed]
azurerm_virtual_network.vnet: Destruction complete after 12s
azurerm_resource_group.rg: Destroying... [id=/subscriptions/cb514673-1b19-46b0-b454-5b670918d9e4/resourceGroups/tp-azure-rg]
azurerm_resource_group.rg: Still destroying... [id=/subscriptions/cb514673-1b19-46b0-b454-5b670918d9e4/resourceGroups/tp-azure-rg, 00m10s elapsed]
azurerm_resource_group.rg: Still destroying... [id=/subscriptions/cb514673-1b19-46b0-b454-5b670918d9e4/resourceGroups/tp-azure-rg, 00m20s elapsed]
azurerm_resource_group.rg: Destruction complete after 23s
Destroy complete! Resources: 18 destroyed.
C:\Projects\tp-terraform-azure>
```

Conclusion

Ce TP a permis de mettre en pratique les fondamentaux de Terraform sur Azure en déployant une infrastructure complète comprenant 2 machines virtuelles Ubuntu avec Nginx, un Load Balancer public, un réseau virtuel sécurisé par un NSG et toutes les ressources associées.

Les principales difficultés rencontrées ont été :

- La région West Europe étant restreinte pour le compte étudiant, il a fallu basculer vers France Central.
- Le Load Balancer Standard ne permettant pas d'accès SSH direct sur les VMs sans IP publique sur les NICs, des IPs publiques temporaires ont été ajoutées pour le débogage puis supprimées.

L'ensemble du code est disponible sur GitHub :
<https://github.com/GeorgeKhita/TP-Terraform>